Event-Driven Programming Cheat Sheet

CS4040 · Unit 3

Pygame Zero event hooks · buttons and keys · clock

Mouse clicks: `on_mouse_down()`

A function that is called when a mouse button is pressed. `pos` is a tuple `(x, y)` with the coordinate where the mouse clicked; `button` is the mouse button that was pressed. You must store `pos` in a global variable to use it in `update()` or `draw()`.

```
def on_mouse_down(pos, button):  # called when a mouse button is pressed
    global click_pos  # so update() and draw() can use the click
    if button == mouse.LEFT:  # which button was pressed
        click_pos = pos  # (x, y) where the mouse clicked
```

Mouse buttons: `mouse.<BUTTON_NAME>`

Constants that represent the buttons on the mouse. Use them to compare with the `button` that was pressed inside `on_mouse_down()`.

```
mouse.LEFT  # left button
mouse.MIDDLE  # middle button
mouse.RIGHT  # right button
mouse.WHEEL_UP  # scroll up
mouse.WHEEL_DOWN  # scroll down
```

Key pressed: `on_key_down()`

Function called when a key is pressed. `key` represents the key that was pressed. This happens once for each time the key is pressed. Event-based!

```
def on_key_down(key):  # called once per key press
    if key == keys.LEFT:  # the left key was pressed
        pass  # do action when left key pressed
```

Key released: `on_key_up()`

Function called when a key is released. `key` represents the key that was released. This happens once for each time the key is released. Event-based!

```
def on_key_up(key):  # called once per key release
    if key == keys.LEFT:  # the left key was released
        pass  # do action when left key released
```

Key constants: `keys.<KEY_NAME>`

Constants that represent keys on the keyboard. Use them in an event listener (not `update()`!) to compare with `key`. See the rest.

```
keys.LEFT  # left arrow key
keys.A  # A key
keys.SPACE  # space bar
keys.ESCAPE  # Esc key
```

Keyboard polling: `keyboard.<key_name>`

A Boolean expression (`True`/`False`) that tells you if the key is currently pressed. Use it in `update()` or in your own functions. This happens as long as the key remains pressed. Polling!

```
def update():
    if keyboard.left:  # True while the left key is pressed
        pass  # do something
    if keyboard.right:  # True while the right key is pressed
        pass  # do something
```

Events vs. polling

	Event-based input	Polling input
Idea	something happens ⟶ you react	you keep checking if something happened
Exp	Text notification when phone on loud. Interrupts.	Notifications silenced. You have to check your phone.
Game design	mouse input, keyboard input, timer event	checking `keyboard` in `update()`
Use case	something should happen **once**, as an immediate response to an action (e.g. jump button pressed)	something should **continue** to happen in response to an action (e.g. move while arrow pressed)

```
def on_key_down(key):
    if key == keys.UP:  # up key pressed
        player.y -= 10  # once per press
```

```
def update():
    if keyboard.d:  # D key held down
        player.x += 1  # every frame
```

start()

- Automatically called before the game begins: place code here to run once at the beginning.
- Used to set up the initial state.
- Useful later when we restart our games to reset the state and play again!

```
def start():  # runs once before draw()/update()
    global countdown_timer
    robot.pos = 250, 250  # reset position
    countdown_timer = 5  # reset timer
```

Callback functions

Callback functions are functions you will define that will be called when a clock event occurs.

We pass just their name (no parentheses) to the `clock.schedule()` calls. Callbacks can not have arguments.

```
# scheduled callback for moving character to random position
def move_character():
    robot.x = random.randint(50, WIDTH - 50)
    robot.y = random.randint(50, HEIGHT - 50)
```

Place the `clock.schedule()` calls elsewhere in the code, but be careful where you put them! If you place one somewhere that gets executed every frame, you may end up scheduling a rapidly growing number of callbacks and crash your program. Placing it in `start()` triggers it once at the beginning:

```
def start():
    robot.pos = (WIDTH // 2, HEIGHT // 2)
    clock.schedule_interval(move_character, 2.0)  # every 2 seconds
```

clock.schedule_unique(callback, delay)

Waits `delay` seconds, then calls `callback`. It cancels any previously scheduled calls to that callback function, so it only will execute once until it is scheduled again.

```
clock.schedule_unique(trex_jump, 0.5)  # call trex_jump in 0.5 s, just once
```

clock.schedule(callback, delay)

Waits `delay` seconds, then calls `callback`. This adds the scheduled callback to any other calls of it already scheduled.

```
clock.schedule(trex_jump, 0.5)  # also call trex_jump in 0.5 s
```

clock.schedule_interval(callback, interval)

Calls `callback` every `interval` seconds. Be careful to call `schedule_interval()` only once for a specific callback function. Otherwise, you will schedule additional intervals!

```
clock.schedule_interval(spawn_coin, 3.0)  # call spawn_coin every 3 seconds
```

clock.unschedule(callback)

```
clock.unschedule(trex_jump)  # cancel all scheduled calls to trex_jump
```

clock.clear()

```
clock.clear()  # unschedule everything
```